

Resumen del proyecto.

En la actualidad, el acceso a la información sobre eventos de ocio se encuentra disperso en múltiples plataformas, lo que dificulta a los usuarios descubrir actividades de forma rápida y eficiente. La mayoría de soluciones existentes presentan la información mediante listados o búsquedas tradicionales, lo que limita la comprensión de la distribución geográfica de los eventos y complica la toma de decisiones.

Ante esta situación, surge Scena como un proyecto orientado a mejorar la forma en la que los usuarios interactúan con la oferta de ocio local. La propuesta se basa en ofrecer una experiencia más visual e intuitiva, permitiendo explorar eventos de manera dinámica y facilitando el acceso a la información relevante.

El proyecto se centra en el ámbito local, concretamente en Mallorca, con el objetivo de validar la solución en un entorno controlado y real. Este enfoque permite analizar el comportamiento de los usuarios, detectar posibles mejoras y sentar las bases para una futura expansión.

Además, Scena se plantea como una aplicación web completa que integra tanto el desarrollo de la interfaz como la lógica de negocio y la gestión de datos. De esta forma, no solo se busca ofrecer una solución funcional, sino también mejorar la experiencia de usuario mediante una herramienta clara, accesible y adaptada a las necesidades actuales.

1.- Descripción del proyecto.

1.1.- ¿Qué es Scena?

Scena es una aplicación web de tipo SPA (Single Page Application) que permite a los usuarios descubrir y consultar eventos de ocio a través de un mapa interactivo. La plataforma muestra eventos geolocalizados, permitiendo al usuario visualizar de forma rápida su ubicación, así como acceder a información detallada como el tipo de evento, horario y descripción. Además, incorpora un sistema de filtrado que facilita la búsqueda de eventos según diferentes criterios como la categoría, la fecha o la localización.

Así, Scena centraliza la información de eventos en una única herramienta, mejorando la experiencia de usuario frente a soluciones tradicionales basadas en listas o búsquedas manuales.

1.2.- Justificación del proyecto.

Scena surge ante la necesidad de mejorar la forma en la que los usuarios descubren eventos de ocio, especialmente en zonas con alta actividad turística como Mallorca, donde la oferta de eventos es amplia pero se encuentra dispersa en diferentes plataformas.

En el análisis previo se identificó que la mayoría de aplicaciones existentes muestran la información en formato de listas o buscadores tradicionales, lo que dificulta la localización rápida de eventos en función de su ubicación geográfica y reduce la claridad en la toma de decisiones por parte del usuario.

Además, en un contexto donde el uso de dispositivos móviles y herramientas de geolocalización está ampliamente integrado en la vida cotidiana, existe una oportunidad clara para mejorar la forma en la que se presenta esta información, priorizando la visualización sobre mapas interactivos.

Por ello, Scena propone una solución centrada en la representación de eventos sobre un mapa de Mallorca, permitiendo al usuario localizar rápidamente actividades cercanas y aplicar filtros según sus intereses. De este modo, se optimiza el proceso de descubrimiento de eventos, ofreciendo una alternativa más directa y visual frente a plataformas tradicionales.

1.3.- Alcance del proyecto.

El alcance del proyecto define los límites funcionales, técnicos y geográficos del sistema, centrándose en el desarrollo de una primera versión funcional de Scena, limitada a la isla de Mallorca. Esta decisión permite validar la idea en un entorno controlado, con una base de datos de eventos localizada y coherente con el objetivo del sistema.

El proyecto contempla el desarrollo completo de una aplicación web, incluyendo el frontend, el backend y la base de datos, así como la integración entre todos los componentes del sistema para garantizar su correcto funcionamiento.

Funcionalidades incluidas en la primera versión:

- Visualización de eventos mediante un mapa interactivo.

- Consultar información detallada de cada evento.
- Sistema de filtrado por categorías, fecha y ubicación.
- Gestión de eventos.
- Comunicación entre cliente y servidor mediante el uso de API REST.

Funcionalidades no incluidas en la primera versión:

- Expansión hacia otras ciudades o regiones.
- Integración de sistemas de compra de entradas.
- Funcionalidades sociales como comentarios, seguidores o interacción entre usuarios.
- Sistemas avanzados de recomendación basados en inteligencia artificial.

1.4.- Metodología de trabajo.

El desarrollo del proyecto se ha llevado a cabo siguiendo la metodología Scrum, organizando el trabajo en sprints cortos con roles definidos y reuniones periódicas para revisar avances y ajustar la prioridad de las tareas.

Esta metodología nos permitió planificar las tareas de forma estructurada, distribuyendo responsabilidades entre los integrantes del equipo y priorizando funcionalidades según su relevancia. También nos facilitó la detección temprana de posibles problemas y permitió mantener un flujo de trabajo ágil y eficiente a lo largo del desarrollo.

Cada sprint incluía planificación, desarrollo, revisión y retrospectiva, asegurándonos que todo avanza de forma organizada y que todas las funcionalidades se implementarán cumpliendo los objetivos establecidos.

2.- Objetivos

2.1.- Objetivo general.

El objetivo general es desarrollar una aplicación web que permita a los usuarios descubrir, consultar y localizar eventos de ocio en Mallorca mediante el uso de un mapa interactivo, facilitando la visualización geográfica de la información y su acceso desde una única plataforma.

2.2.- Objetivos específicos.

2.2.1.- Objetivos técnicos.

Los objetivos técnicos del proyecto se centran en establecer una arquitectura sólida, escalable y mantenible para el desarrollo de la aplicación. Estos objetivos se han materializado mediante las siguientes implementaciones:

- Diseñar e implementar una arquitectura cliente-servidor separando el frontend y el backend, permitiendo de esta manera una mayor organización del código y facilitando su escalabilidad y mantenimiento.
- Implementar la comunicación entre el frontend y el backend mediante una API REST, utilizando el intercambio de datos en formato JSON para permitir una comunicación estructurada y eficiente entre ambas capas.

- Integrar una base de datos relacional para el almacenamiento de la información del sistema, modelando las entidades como usuarios, eventos y categorías, asegurando la coherencia de los datos mediante el uso de relaciones y restricciones.
- Implementar mecanismos de seguridad en el backend, incluyendo el uso de autenticación basada en tokens, encriptación de contraseñas y validación de datos de entrada para proteger el acceso al sistema.

2.2.2.- Objetivos funcionales.

Los objetivos funcionales definen las capacidades que ofrece la aplicación al usuario final. Estos son los siguientes:

- Permitir la visualización de eventos mediante un mapa interactivo, mostrando su localización geográfica exacta a través de marcadores.
- Representar cada evento con información estructurada, incluyendo su nombre, categoría, ubicación, fecha, horario y descripción.
- Implementar un sistema de filtrado el cual permite a los usuarios buscar eventos según criterios definidos, como la categoría del evento, fecha y localización.
- Garantizar la correcta visualización y funcionamiento de la aplicación en varios dispositivos, como ordenadores, dispositivos móviles y tablets, mediante el uso de un diseño responsive.

2.2.3.- Enfoque en la UI y UX.

El diseño de la interfaz y la experiencia de usuario han sido aspectos clave durante el desarrollo del proyecto, ya que influyen directamente en la usabilidad y en la facilidad de interacción con la aplicación.

Para ello, se han tomado las siguientes decisiones sobre el diseño:

- Uso de un mapa interactivo como elemento principal de navegación.
- Diseño de una interfaz limpia y simplificada, reduciendo así la sobrecarga de información y eliminando la información irrelevante.
- Representación de los eventos en tarjetas detalladas, facilitando la comprensión de forma más rápida.
- Adaptación de la interfaz a distintos dispositivos mediante un diseño responsive.

3.- Contextualización.

3.1.- Descripción de la empresa.

La empresa Scena S.L. Se centra en el desarrollo de plataformas web orientadas a la gestión de eventos de ocio. El enfoque de la empresa se basa en la creación de aplicaciones que integran tanto la parte visual como la lógica de negocio, permitiendo ofrecer soluciones completas y escalables. En este caso el producto desarrollado es una aplicación web, la cual facilita la exploración de eventos mediante un mapa, mejorando la experiencia del usuario frente a soluciones tradicionales.

La empresa está formada por un equipo, actualmente de solo tres integrantes, lo que permite una comunicación directa, una toma de decisiones ágil y una mayor flexibilidad a la hora de desarrollar el proyecto. Los principales roles del equipo son los siguientes:

- Responsable de planificación.
- Desarrollador frontend.
- Desarrollador backend.

Datos generales de la empresa:

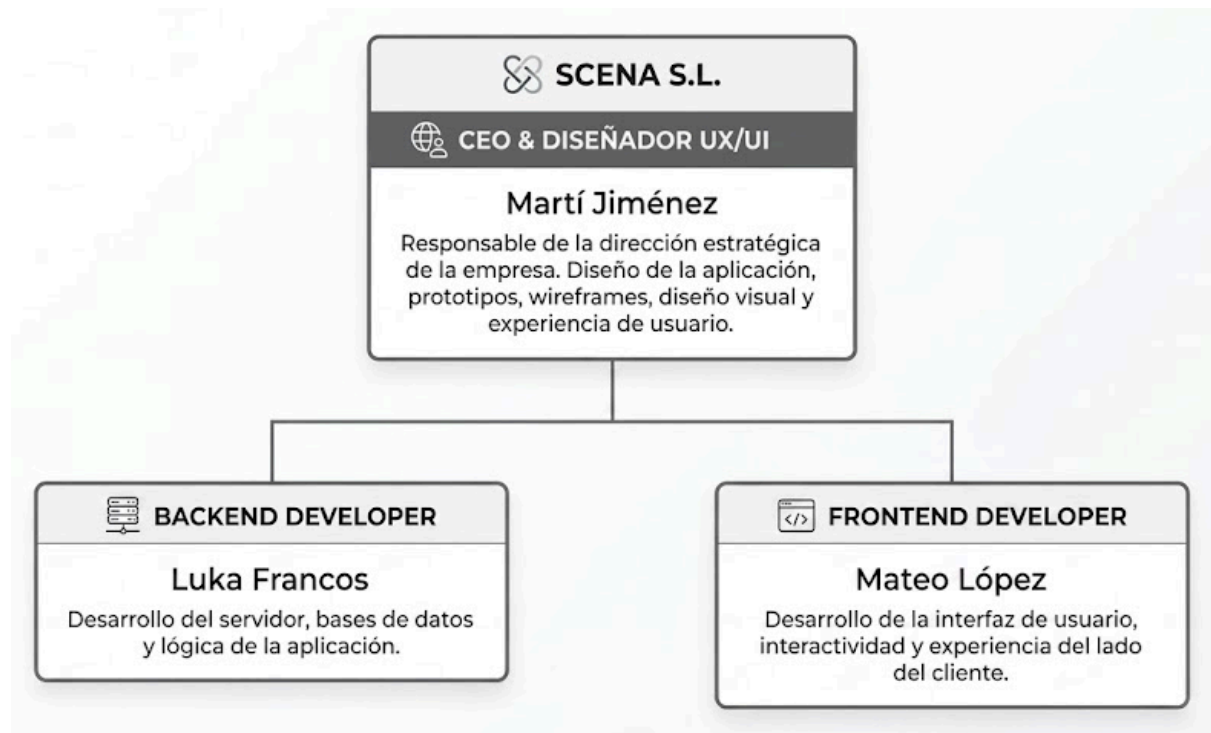
- Año de creación: 2025
- Sede principal: CIFP Francesc de Borja Moll
- Número de empleados: 3
- Sector: desarrollo de software

3.2 .- Organigrama.

La estructura de la empresa es sencilla y está orientada a que el flujo de trabajo sea óptimo, además de mejorar la coordinación entre los miembros del equipo.

- CEO → Martí Jiménez
 - Responsable de la planificación, definición de requisitos y la organización de las tareas.
- Backend developer → Luka Francos
 - Encargado de desarrollar toda la parte de la lógica de negocio, desarrollo de las APIs REST y gestión de la BBDD.
- Frontend developer → Mateo López
 - Responsable de la interfaz y experiencia de usuario, además de la implementación de los componentes visuales de la web.
- Diseñador UX/UI → Martí Jiménez
 - Encargado del diseño de la aplicación, incluyendo los prototipos, wireframes, diseño visual y la experiencia de usuario.

Aunque exista esta jerarquía, todos los integrantes del equipo trabajan en todos los sectores para poder ayudar al desarrollo de la aplicación.



3.3.- Descripción del departamento.

El departamento de desarrollo es el encargado de llevar a cabo la implementación completa del proyecto, abarcando todas las fases propuestas a lo largo del proyecto.

Podemos encontrar algunas de las funcionalidades a continuación:

- Análisis de requisitos y definición de funcionalidades.
- Diseño de la arquitectura del sistema.
- Desarrollo de la aplicación web, tanto la parte visual como funcional.
- Gestión y diseño de la BBDD.
- Integración de servicios y APIs.
- Corrección de errores.

El trabajo realizado dentro del departamento se organiza siguiendo una metodología ágil, la cual permite adaptar el desarrollo a las necesidades del proyecto, lo cual ofrece el realizar mejoras de forma continua.

3.4.- Descripción de perfiles.

Los perfiles que nos podemos encontrar en el proyecto presentan roles diferenciados pero complementarios, lo cual permite cubrir todas las áreas necesarias para el desarrollo de la aplicación.

Backend developer:

Es el encargado de la lógica de negocio y el funcionamiento interno de la aplicación.

Las principales responsabilidades incluyen:

- Desarrollo de APIs REST para poder comunicarse con el frontend.
- Gestión de la base de datos y modelado de las entidades.

- Implementación de la seguridad del sistema (tokens y autenticaciones).

Conocimientos necesarios:

- Spring Boot.
- JPA/Hibernate.
- PostgreSQL.

Frontend developer:

Responsable de la parte visual y la interacción con el usuario. Las principales funciones son:

- Desarrollo de la interfaz de usuario.
- Implementación de componentes y vistas.
- Integración con el backend mediante el consumo de APIs.
- Mejora de la experiencia de usuario.

Conocimientos necesarios:

- Angular.
- TypeScript.
- HTML.
- CSS.
- Bootstrap.

CEO / Product Owner:

Encargado de la organización del proyecto y la gestión de tareas. Sus funciones principales son:

- Definición de requisitos funcionales.
- Planificación del desarrollo.
- Priorizar tareas.
- Supervisión del progreso.

Diseñador UI/UX:

Responsable de la apariencia visual y la experiencia de usuario de la aplicación. Las responsabilidades principales son:

- Creación de prototipos, wireframes y mockups.
- Definición de flujos de navegación claros e intuitivos.
- Diseño visual de la interfaz y elementos gráficos.
- Colaboración con frontend para garantizar la implementación del diseño.

Conocimientos necesarios:

- Herramientas de diseño: figma, sketch, etc.
- Conocimiento sobre los principios de la UI/UX.

3.5.- Riesgos laborales.

Existen varios riesgos laborales asociados al trabajo prolongado frente a las pantallas y al uso continuo de equipos informáticos. Estos riesgos afectan principalmente a la salud física y mental de los desarrolladores si no se aplican diversas medidas preventivas adecuadas. Los principales riesgos que se han identificado son los siguientes:

- Fatiga visual: esta es causada por la exposición prolongada frente a pantallas, lo cual provoca sequedad ocular, visión borrosa, molestias o somnolencia.
- Dolores musculares y problemas en las posturas: estos son derivados por una mala ergonomía en el puesto de trabajo, especialmente en la espalda, cuello, hombros y muñecas.
- Sobrecarga mental o estrés: debido a una alta concentración requerida en las tareas.
- Lesiones por movimientos repetitivos: estas lesiones pueden aparecer por hacer un uso continuado del teclado y del ratón, los cuales pueden derivar en problemas como el síndrome del túnel carpiano.
- Sedentarismo: es provocado por largas jornadas de trabajo sin actividad física, lo cual puede afectar a la salud en general.
- Aislamiento social: sobre todo en entornos de trabajo remoto o donde exista poca interacción social.

Hay varias medidas preventivas a tener en cuenta para minimizar los riesgos identificados:

- Realizar pausas periódicas para descansar la vista y evitar la fatiga visual, como la regla 20-20-20 por ejemplo.
- Mantener una correcta ergonomía en el puesto de trabajo, ajustando la altura de las pantallas, silla y mesa.
- Hacer uso de sillas ergonómicas y apoyar en ella correctamente la espalda.
- Alternar periodos de trabajo con descansos para reducir la carga mental.
- Realizar estiramientos y varios ejercicios físicos durante la jornada laboral.
- Mantener una correcta iluminación en el entorno de trabajo, evitando de esta manera los reflejos en la pantalla.

4.- Análisis estratégico.

4.1.- Modelo de negocios

Los objetivos a nivel estratégico buscan definir el posicionamiento del proyecto y su posible evolución en un entorno real con varios usuarios.

- Posicionar Scena como una herramienta diferenciadora dentro del sector, destacando el uso del mapa como elemento principal en la navegación y descubrimiento de eventos.
- Ofrecer una plataforma útil tanto para los usuarios con intención de asistir a los eventos, como para los creadores de los eventos, facilitando la visualización de eventos mediante la localización.

- Explorar posibles modelos de monetización como la inclusión de publicidad o simplemente una comisión por la redirección de usuarios hacia la compra de entradas.

4.2.- Estrategia de mercado.

Scena es una plataforma orientada al descubrimiento de eventos locales mediante una experiencia visual basada en mapas interactivos. La estrategia de mercado se ha centrado en diferenciarse de otras plataformas existentes a través de la facilidad de uso, la claridad visual y la personalización de la experiencia de usuario que ofrecemos.

A diferencia de otras soluciones más tradicionales, Scena apuesta libremente por una navegación más intuitiva, donde los usuarios pueden identificar rápidamente los eventos según su ubicación geográfica, reduciendo el tiempo de búsqueda y mejorando de esta manera la toma de decisiones.

El público objetivo principal está formado por:

- Usuarios finales: jóvenes y adultos interesados en el ocio nocturno, eventos culturales y actividades sociales.
- Promotores con locales: negocios que desean dar visibilidad a sus eventos y atraer un público segmentado.

Estrategia de lanzamiento:

La estrategia inicial se basa en un lanzamiento local en Palma de Mallorca, lo cual permite de esta forma validar la idea en un entorno real controlado y obtener de esta manera un feedback real de los usuarios. Este enfoque facilita:

- Detectar posibles mejoras en la plataforma.
- Ajustar funcionalidades según el comportamiento del usuario.
- Optimizar la experiencia antes de escalar el producto.

Estrategia de crecimiento:

Una vez el producto se haya validado mediante la estrategia de lanzamiento, se plantean las siguientes líneas de crecimiento:

- Expansión geográfica a otras ciudades o regiones.
- Incorporación de nuevas funcionalidades, ya sea sociales, recomendaciones, etc.
- Mejora continua de la experiencia de usuario basada en datos reales.

Estrategia de captación de usuarios:

Para poder atraer nuevos usuarios y promotores, se plantean las siguientes acciones:

- Promoción en redes sociales.
- Colaboración con locales y organizadores de eventos.
- Difusión en entornos locales, ya sea universidades, zonas de ocio, etc.

Generación de valor:

El objetivo principal es generar valor tanto para usuarios como para promotores:

- Usuarios: acceso rápido, claro y visual a la información detallada de los eventos.
- Promotores: mayor visibilidad y capacidad de atraer público objetivo.

De esta manera, Scena se posicionaría como una herramienta útil que mejoraría la conexión entre la oferta y la demanda dentro del sector del ocio.

4.3.- Plan financiero.

El plan financiero se basa en una estimación de los costes iniciales y operativos del proyecto, así como en la identificación de posibles fuentes de ingresos a medio y largo plazo. Tratándose de una primera versión, se ha optado simplemente por una estrategia de bajo coste la cual permitirá validar la viabilidad del producto antes de realizar inversiones mayores.

Costes estimados:

Infraestructura:

Los costes asociados a la infraestructura corresponden a los servicios necesarios para garantizar el correcto funcionamiento, almacenamiento y acceso a la aplicación en un entorno real. Se ha optado por soluciones en la nube y servicios gestionados, lo cual permite reducir la complejidad técnica y asegurar una mayor disponibilidad del sistema. Los costes son los siguientes:

- Servidor backend (VPS): 40~50€/mes.
- Base de datos (Supabase): 20€/mes.
- Hosting frontend (Vercel): 15€/mes.

Herramientas de desarrollo:

Las herramientas utilizadas durante el desarrollo del proyecto son en su mayoría gratuitas o de código abierto, lo cual permite reducir significativamente los costes sin afectar a la calidad del producto final. Además, facilitan el trabajo colaborativo y la gestión eficiente del código:

- Entorno de desarrollo: Visual Studio Code.
- Herramientas de diseño: Figma.
- Contenerización y despliegue local: Docker Desktop.
- Control de versiones: Git y GitHub,

Teniendo en cuenta los costes sobre la infraestructura y las herramientas de desarrollo se quedaría en aproximadamente 80~90€/mes.

Costes de recursos humanos.

Los costes de recursos humanos representan la mayor inversión en un entorno profesional, ya que el desarrollo, mantenimiento y evolución del producto dependen

directamente del equipo técnico. El equipo técnico que nos podemos encontrar es el siguiente:

- Backend developer: 3.000~3.500€/mes.
- Frontend developer: 3.000~3.500€/mes.
- Product owner: 3.000~4.000€/mes.

El coste total aproximado del equipo es de 9.000~11.000€/mes.

Inversión inicial.

La inversión inicial del proyecto es reducida, ya que se basa principalmente en el uso de herramientas gratuitas y servicios con bajo coste mensual, lo cual permite iniciar el desarrollo del proyecto sin la necesidad de una gran financiación/inversión.

Fuentes de ingresos.

El modelo de negocio contempla varias vías de monetización que pueden implementarse progresivamente a medida que crece la plataforma y la base de usuarios. Estás son:

- Promoción de eventos: locales y organizadores pueden pagar por destacar sus eventos dentro de la plataforma, aumentando de esta manera su visibilidad.
- Publicidad dentro de la plataforma: inserción de anuncios relacionados con el ocio, eventos o locales.
- Modelo freemium: ofrecer funcionalidades básicas gratuitas y opciones avanzadas de pago para los organizadores, como las estadísticas, etc.
- Comisiones por la venta de entradas: posibilidad de integrar un sistema de venta/redirección a los portales de venta de entradas, obteniendo una comisión por cada entrada vendida.

Visibilidad económica.

El proyecto presenta una alta viabilidad en su fase inicial, ya que los costes de infraestructura son bajos y no requiere una inversión inicial elevada. Además, al hacer uso de tecnologías open source permite reducir gastos sin comprometer la calidad. El modelo de ingresos planteado permite generar beneficios de forma progresiva, en función del crecimiento de usuarios y de la adopción por parte de los organizadores de eventos.

Escalabilidad económica.

A medida que el proyecto evolucione, será necesario adaptar la infraestructura y los recursos humanos a la demanda del sistema. Esto puede implicar:

- Un aumento de capacidad de servidores.
- Optimización del rendimiento.
- Ampliación del equipo de desarrollo.

4.4.- Análisis de la competencia.

Analizar a la competencia nos permite identificar las fortalezas y debilidades de varias plataformas similares, así como definir el valor diferencial de Scena dentro del mercado. Este análisis permite detectar oportunidades de mejora y establecer una propuesta diferenciadora basada en la experiencia de usuario (UX).

4.4.1.- Eventbrite - Competencia A

Eventbrite es una plataforma consolidada a nivel internacional centrada en la gestión, promoción y venta de entradas para eventos. Está orientada principalmente a organizaciones que desean gestionar todo lo relacionado con el evento.

Características principales:

- Creación y gestión completa de eventos.
- Venta de entradas online con pasarela de pago integrada.
- Sistema de búsqueda por categorías, ubicación y fechas.
- Herramientas de marketing y promoción.
- Gestión de asistentes y control de aforo.

Ventajas:

- Plataforma muy consolidada y de confianza.
- Amplia oferta de eventos a nivel global.
- Funcionalidades avanzadas para organizadores.
- Integración con múltiples herramientas externas.

Desventajas:

- Interfaz basada principalmente en listas.
- Falta de enfoque en la visualización geográfica.
- Experiencia menos intuitiva para explorar eventos rápidamente.
- Costes asociados a los organizadores.

4.4.2.- Meetup - Competencia B.

Meetup es una plataforma orientada a la creación de comunidades y organización de eventos basados en intereses comunes. Su enfoque principalmente es fomentar la interacción social entre todos los participantes.

Características principales:

- Creación de grupos y comunidades.
- Organización de eventos.
- Sistema de recomendaciones basado en los intereses de cada usuario.
- Interacción entre usuarios, ya sea desde comentarios, asistencia o la participación de ellos en los eventos.

Ventajas:

- Fuerte componente social.
- Sistema de recomendación personalizado.
- Gran base de usuarios activos.
- Facilita la creación de comunidades.

Desventajas:

- Menor enfoque en la exploración de eventos.
- Interfaz centrada en grupos que grupos, no tanto en eventos.
- Visualización de geográfica muy poco destacada.

4.4.3.- Google Events - Competencia C.

Google Events viene integrado dentro de Google Search y Google Maps, lo cual permite a los usuarios descubrir los eventos cercanos a través de búsquedas o navegación geográfica. Los datos los recibe de varias fuentes externas.

Características principales:

- Integración con buscador y mapas
- Visualización de eventos cercanos automáticamente
- Información agregada desde distintas plataformas
- Acceso rápido sin necesidad de registro

Ventajas:

- Alta accesibilidad (uso directo desde Google)
- Integración con mapas y geolocalización
- Gran volumen de datos disponibles
- Experiencia rápida y sin fricción

Desventajas:

- Falta de personalización avanzada
- Información dependiente de fuentes externas (no siempre completa)
- No permite gestión directa de eventos desde una plataforma propia
- Interfaz poco especializada en eventos

Tabla comparativa entre los distintos competidores y Scena.

Característica	Eventbrite	Meetup	Google Events	Scena
<u>Visualización en mapa</u>	NO	LIMITADO	SI	SI
<u>Venta de entradas</u>	SI	NO	NO	NO
<u>Creación de eventos</u>	SI	SI	NO	SI
<u>Enfoque social</u>	NO	SI	NO	NO
<u>Personalización mediante filtros</u>	SI	SI	LIMITADO	SI
<u>Facilidad de uso</u>	LIMITADO	LIMITADO	SI	SI

<u>Enfoque geográfico</u>	LIMITADO	LIMITADO	SI	SI
<u>Especialización en eventos</u>	SI	SI Y NO	NO	SI

4.5.- Análisis DAFO.

FORTALEZAS • Desarrollo full-stack realizado por el propio equipo. • Uso de tecnologías modernas como Angular, Spring Boot y PostgreSQL. • Interfaz basada en mapa interactivo que mejora la experiencia de usuario. • Arquitectura cliente-servidor bien estructurada y separada por capas. • Diseño centrado en UX, priorizando claridad y facilidad de uso.	DEBILIDADES • Tamaño reducido del equipo, lo que limita el trabajo en paralelo. • Falta de experiencia en entornos de producción a gran escala o despliegues complejos. • Alcance limitado únicamente a la isla de Mallorca. • Capacidad de mantenimiento limitada ante actualizaciones masivas.
OPORTUNIDADES • Crecimiento del sector de ocio y eventos locales en zonas turísticas. • Aumento del uso de aplicaciones basadas en geolocalización. • Tendencia hacia plataformas digitales personalizadas. • Posibilidad de expansión a otras ciudades o regiones. • Interés de promotores en nuevas formas de visibilidad digital.	AMENAZAS • Alta competencia de plataformas consolidadas (Eventbrite, Meetup, Google Events). • Dependencia de servicios externos (mapas, hosting, APIs). • Dificultad para diferenciarse en un mercado consolidado. • Baja visibilidad inicial frente a grandes plataformas.

4.5.1.- Factores internos.

Los factores internos hacen referencia a aquellos elementos/factores que dependen directamente del equipo de desarrollo y de las decisiones tomadas durante la planificación e implementación del proyecto. Estos factores permiten identificar tanto los puntos fuertes del proyecto como sus limitaciones actuales.

Fortalezas.

El proyecto presenta una serie de fortalezas las cuales aportan valor tanto a nivel técnico como funcional:

- Desarrollo full-stack realizado por el propio equipo, abarcando todas las partes del proyecto, backend, frontend y base de datos.
- Uso de tecnologías modernas y ampliamente utilizadas en la industria, como Angular, Spring Boot y PostgreSQL.
- Implementación de una interfaz basada en mapa interactivo, lo que mejora significativamente la experiencia de usuario frente a soluciones tradicionales basadas en listas.
- Arquitectura cliente-servidor bien estructurada y separada por capas, lo cual facilita la escalabilidad y el mantenimiento entre los miembros.
- Diseño centrado en la experiencia de usuario (UX), priorizando la claridad visual y la facilidad de uso.

Debilidades.

A pesar de todas las fortalezas que nos encontramos, el proyecto también presenta ciertas limitaciones propias de su fase de desarrollo y del contexto del equipo:

- Tamaño reducido del equipo, lo que limita la capacidad de trabajo en paralelo.
- Falta de experiencia previa en entornos de producción a gran escala o despliegues complejos.
- Alcance geográfico limitado únicamente a la isla de Mallorca en la versión actual del proyecto.

4.5.2.- Factores externos.

Los factores externos hacen referencia a aquellos elementos que no dependen directamente del equipo de desarrollo, pero que pueden influir de forma positiva o negativa en la evolución, adopción y crecimiento del proyecto. Estos factores están relacionados con el entorno tecnológico, el mercado y la competencia.

Oportunidades.

El proyecto se encuentra en un entorno con varias oportunidades de crecimiento y expansión:

- Crecimiento del sector del ocio y eventos locales, especialmente en zonas turísticas como Mallorca.
- Aumento del uso de aplicaciones basadas en geolocalización y mapas interactivos. Tendencia creciente hacia plataformas digitales personalizadas y centradas en la experiencia de usuario.
- Posibilidad de expansión del proyecto a otras ciudades o regiones en el futuro.
- Interés de promotores y locales en nuevas formas de visibilidad digital para sus eventos.
- Avances en tecnologías web que facilitan la escalabilidad y mejora continua de la aplicación.

Amenazas.

El proyecto también se enfrenta a una serie de amenazas externas que pueden afectar su posicionamiento o crecimiento:

- Alta competencia en el sector, con plataformas consolidadas como Eventbrite, Meetup o Google Events.
- Dependencia de servicios externos, como los mapas, el hosting, etc. Esto puede cambiar de precios o condiciones.
- Dificultad para diferenciarse en un mercado ya existente con soluciones ya establecidas.
- Limitaciones iniciales de visibilidad y captación de usuarios en comparación con grandes plataformas.

5.- Planificación.

5.1.- Metodología de planificación.

El proyecto se ha desarrollado siguiendo la metodología Scrum, la cual aporta un enfoque ágil que permite organizar el trabajo en iteraciones cortas las cuales denominamos sprints. Esta metodología facilita la división del proyecto en tareas más pequeñas, las cuales son asignadas a cada miembro del equipo, permitiendo de esta manera un seguimiento continuo del progreso.

Cada sprint se estructura de las siguientes fases:

- Planificación: definición de tareas, objetivos y prioridades a corto plazo.
- Desarrollo: implementación de las funcionalidades planificadas.
- Revisión: validaciones de las funcionalidades desarrolladas.
- Retrospectiva: análisis del trabajo realizado para detectar posibles mejoras en el siguiente ciclo.

El uso de esta metodología ha permitido mantener una organización clara sobre el trabajo, mejorando la coordinación entre los miembros del equipo y poder adaptar el desarrollo a las necesidades del proyecto.

5.2.- Fases del proyecto.

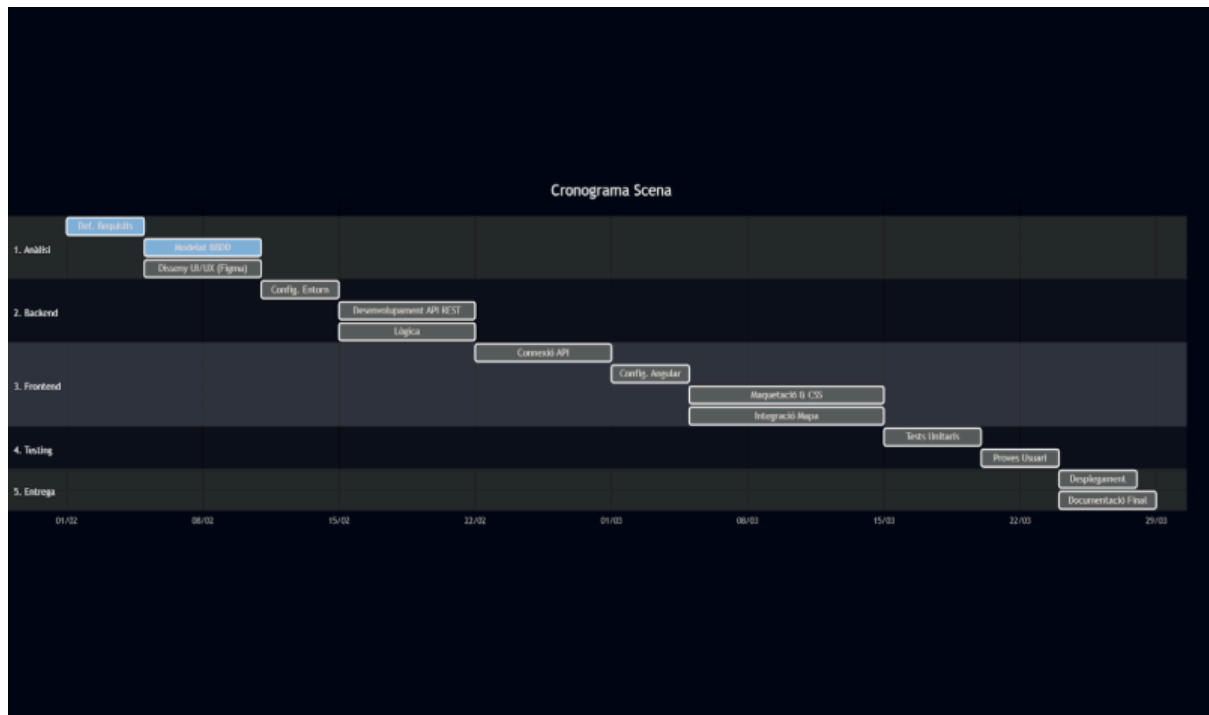
El desarrollo del proyecto se ha estructurado en varias fases que corresponden al ciclo de vida de una aplicación:

1. Análisis de requisitos:
 - Identificación de funcionalidades principales del sistema.
 - Estudio de plataformas similares para poder detectar carencias.
 - Definición de requisitos funcionales y no funcionales.
2. Diseño del sistema:
 - Definición de la arquitectura cliente-servidor.
 - Modelado de la BBDD y relaciones entre las distintas entidades.
 - Elaboración de diagramas UML, como los de clases, casos de uso...
 - Diseño de la interfaz y experiencia de usuario (UI / UX).
3. Desarrollo e implementación:
 - Desarrollo del frontend (interfaz, mapa, filtros...).
 - Desarrollo del backend (API REST, lógica de negocio).
 - Integración entre frontend y backend.
4. Validación:
 - Verificación del correcto funcionamiento de la aplicación.
 - Detección y corrección de errores.
 - Ver la compatibilidad con distintos navegadores y dispositivos.
5. Despliegue:
 - Preparación del entorno para su ejecución.
 - Publicación de la aplicación.

5.3.- Cronograma.

El desarrollo del proyecto se ha organizado mediante un cronograma que permite distribuir las tareas a lo largo del tiempo y visualizar la duración de cada fase.

El cronograma se ha representado mediante un diagrama de Gantt, donde podemos ver que se reflejan diferentes etapas del proyecto, así como la secuencia de estas y posibles solapamientos.



5.4.- Recursos.

5.4.1.- Recursos humanos.

El desarrollo del proyecto ha sido llevado a cabo por solo tres integrantes, lo que ha permitido una comunicación directa y una mayor flexibilidad en la toma de decisiones.

Los roles principales dentro del equipo han sido los siguientes:

- CEO/Product Owner: encargado de la planificación del proyecto, definición de los requisitos y la priorización de tareas. Encargado también de supervisar el avance del desarrollo, asegurando que se cumplen los objetivos establecidos.
- Backend developer: responsable sobre el desarrollo de la lógica de negocio, implementación de la API REST, la gestión de la base de datos y la configuración de la seguridad en el sistema.
- Frontend developer: encargado en el desarrollo de la UI y de la UX, además de integrar el consumo de APIs.

5.4.2.- Recursos técnicos.

Para el desarrollo del proyecto se han utilizado varios recursos técnicos tanto a nivel de hardware como de software:

- Equipos informáticos: ordenadores con capacidad suficiente para ejecutar entornos de desarrollo, servidores locales y herramientas necesarias para completar el desarrollo.

- Entorno de desarrollo: uso de IDEs especializados, en este caso Visual Studio Code, navegadores web para poder realizar pruebas, herramientas como Postman para poder atacar a los endpoints REST y herramientas de debugging.
- Herramientas de desarrollo: frameworks y librerías utilizadas en frontend y backend, en este caso Spring Boot y Angular principalmente.
- Sistema de control de versiones: Git y GitHub son los utilizados para la gestión del código fuente, control de cambios y trabajo colaborativo.
- Base de datos: servidor PostgreSQL alojado en Supabase (en deploy, cuando se trabaja en local se hace con un contenedor de docker) para la gestión de datos.

5.5.- Riesgos de planificación.

Los posibles riesgos de planificación que hemos podido plantearnos son los siguientes:

- Retrasos en el desarrollo: debido a una incorrecta estimación del tiempo necesario para completar una tarea determinada o la aparición de problemas técnicos que no se han previsto.
- Descoordinación del equipo: problemas a la hora de comunicación o la falta de seguimiento del progreso.
- Cambios en los requisitos: modificación de funcionalidades o nuevas necesidades detectadas durante el desarrollo.
- Sobrecarga de trabajo: acumulación de tareas en determinados momentos del proyecto, sobre todo en las fases finales del proyecto.
- Problemas técnicos: errores durante la integración entre el frontend y backend o dificultades con determinadas tecnologías.

Medidas preventivas.

Para minimizar el impacto de estos riesgos, se han aplicado las siguientes medidas:

- Definición clara de tareas y prioridades desde el inicio.
- Organización del trabajo en fases o sprints, facilitando el control del progreso.
- Seguimiento continuo del desarrollo para detectar posibles desviaciones a tiempo.
- Revisión periódica del trabajo realizado.
- Comunicación constante entre los miembros del equipo.

6.- Diseño

6.1.- Lenguaje y herramientas utilizadas.

En este apartado se describen las tecnologías utilizadas a lo largo del desarrollo del proyecto, así como la justificación de la selección de estos en función de los requisitos del sistema.

6.1.1.- Frontend.

El frontend de la aplicación se ha desarrollado utilizando el framework Angular, junto al lenguaje TypeScript, permitiendo de esta manera la construcción de una SPA (Single Page Application) basada en componentes reutilizables.

El uso de Angular facilita la separación de responsabilidades entre los distintos módulos, servicios y componentes que nos podemos encontrar, además de permitir una gestión eficiente del estado y la comunicación con el backend a través de servicios HTTP.

A la hora de desarrollar la interfaz se han utilizado HTML, CSS y Bootstrap, lo cual ha sido esencial para crear una interfaz adaptable a todos los dispositivos. Además de agregar una librería de mapas, concretamente Leaflet, la cual permite representar los eventos mediante marcadores geográficos y gestionar la interacción del usuario con el mapa.

Aspectos destacables:

- Arquitectura basada en componentes.
- Uso de servicios para el consumo de API.
- Separación entre la lógica y la presentación.
- Interfaz responsive

6.1.2.- Backend.

El backend se ha desarrollado sobre Java utilizando el framework de Spring Boot, lo cual permite la creación de una API REST robusta, modular y escalable.

Spring Boot facilita todo el tema de la configuración inicial del proyecto sin la necesidad de configuraciones complejas. Además, permite la creación de endpoints REST de forma sencilla y organizada, siguiendo una arquitectura basada en controladores.

De la gestión de la seguridad se encarga Spring Security, el cual se centra en la autenticación y autorización de usuarios mediante mecanismos seguros, permitiendo controlar el acceso a los distintos apartados del sistema.

En cuanto a la persistencia de datos, se ha elegido hacer uso sobre el servicio JPA junto con hibernate, lo cual permite mapear las entidades del sistema en tablas de la base de datos de forma automática mediante el uso de anotaciones. Esto facilita el trabajo con datos, evitando la necesidad de escribir consultas SQL complejas en la mayoría de los casos.

Además, el backend se ha estructurado siguiendo una arquitectura en capas, diferenciando claramente los siguientes componentes:

- Controladores (Controllers): encargados de gestionar las peticiones HTTP.
- Servicios (Services): donde se implementa la lógica de negocio.
- Repositorios (Repositories): responsables del acceso a la base de datos.

Responsabilidades que debe de abarcar el backend.

- Gestionar la lógica de negocio de la aplicación.
- Validar los datos de entrada recibidos desde el frontend antes de procesarlos.

- Exponer los endpoints REST para la comunicación con el cliente.
- Garantizar la seguridad del sistema y la protección de datos.

Funcionamiento general.

El backend actúa como un intermediario entre el frontend y la base de datos, ya que recibe las peticiones HTTP, las procesa aplicando la lógica de negocio correspondiente devolviendo las respuestas en formato JSON.

6.1.3.- Base de datos.

La base de datos utilizada en el proyecto es PostgreSQL, la cual es una base de datos relacional la cual permite almacenar la información de forma estructurada y garantizar la integridad de los datos mediante el uso de relaciones y restricciones. Se ha diseñado un modelo de datos basado en entidades las cuales representan los elementos principales del sistema.

Modelo de datos.

Las principales entidades definidas en la base de datos son:

- Usuarios: almacenan la información de los usuarios registrados en la aplicación.
- Eventos: contienen los datos relacionados con cada evento, como el nombre, likes recibidos, etc.
- Categorías: permiten clasificar los eventos según su tipo.
- Relaciones entre usuarios y eventos: utilizada para gestionar las interacciones entre estas dos entidades.

Persistencia y acceso a datos.

El acceso a la base de datos se realiza mediante JPA e Hibernate, lo que permite trabajar con objetos en vez de hacer uso de consultas SQL de manera directa.

Las ventajas que nos ofrece hacer el proceso de esta manera son:

- Reducción de código repetitivo-
- Mayor legibilidad.
- Independencia del motor de base de datos.
- Facilidad para realizar las operaciones CRUD.

Infraestructura de la base de datos.

La base de datos está alojada en un entorno gestionado mediante Supabase, lo que nos aporta múltiples ventajas:

- Gestión remota: acceso desde cualquier entorno de desarrollo.
- Backups automáticos: copias de seguridad periódicas para evitar la pérdida de datos.
- Escalabilidad: posibilidad de aumentar recursos según sus necesidades.
- Seguridad: acceso controlado mediante credenciales y configuración de permisos.

Justificación de la elección:

Se ha optado por PostgreSQL frente a varias soluciones debido a:

- Su fiabilidad y estabilidad en entornos de producción.
- Su capacidad para gestionar relaciones complejas.
- Su compatibilidad con herramientas como JPA/Hibernate.
- Su buen rendimiento en aplicaciones web.

6.1.4.- Despliegue y contenerización.

El despliegue del sistema se realiza mediante el uso de Docker, permitiendo de esta manera encapsular la aplicación en contenedores independientes y facilitar su ejecución en diferentes entornos.

Se han definido contenedores para:

- Backend (Spring Boot).
- Base de datos (PostgreSQL, si es de forma local).

Hacer uso de Docker aporta las siguientes ventajas:

- Portabilidad del entorno de ejecución.
- Aislamiento de dependencias.
- Facilidad de despliegue.
- Reproducibilidad del sistema.

La orquestación básica se llevará a cabo mediante Docker Compose, lo cual nos permite levantar todos los servicios del proyecto mediante un único comando.

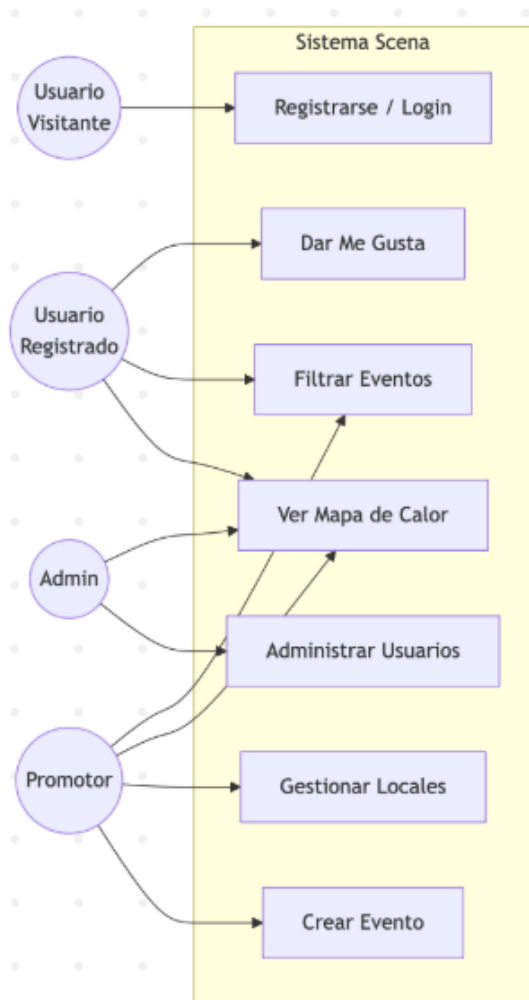
Los servicios definidos son los siguientes:

- Servicio de backend.
- Servicio de BBDD.
- Red interna la cual permite la comunicación entre los servicios.

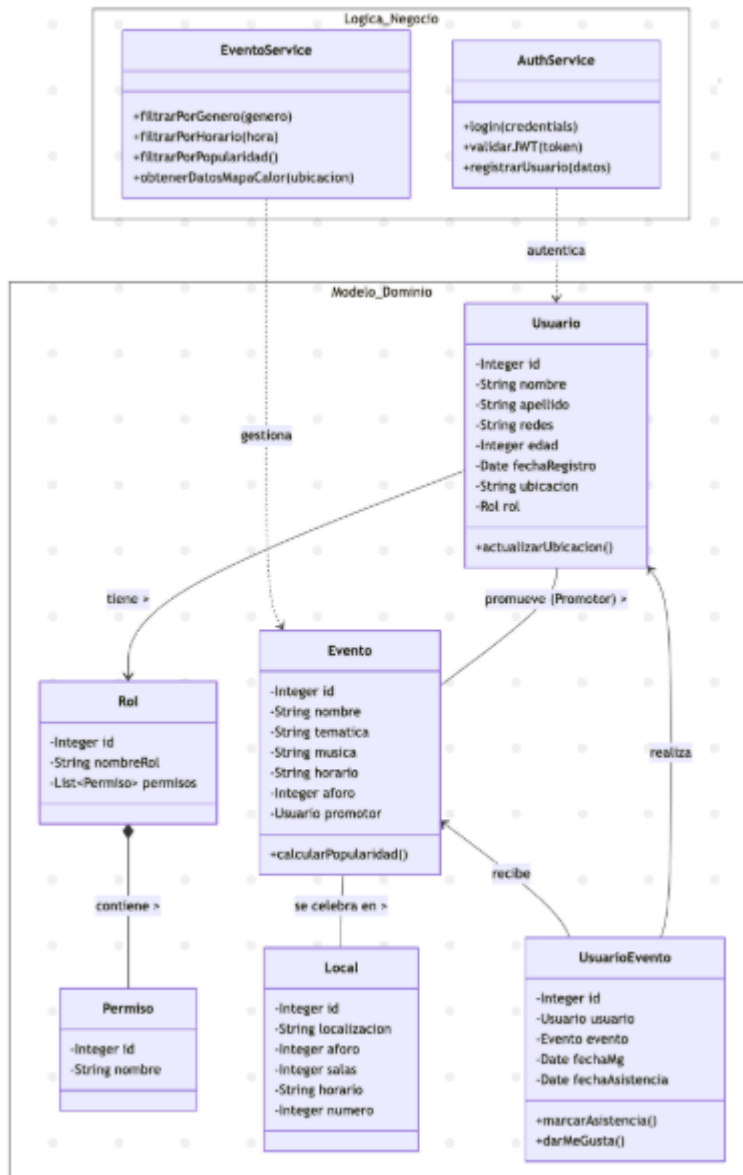
6.2.- Diagramas y modelos.

Los diagramas UML permiten representar el sistema desde varios puntos de vista, los cuales facilitan tanto el diseño como la comprensión de ellos.

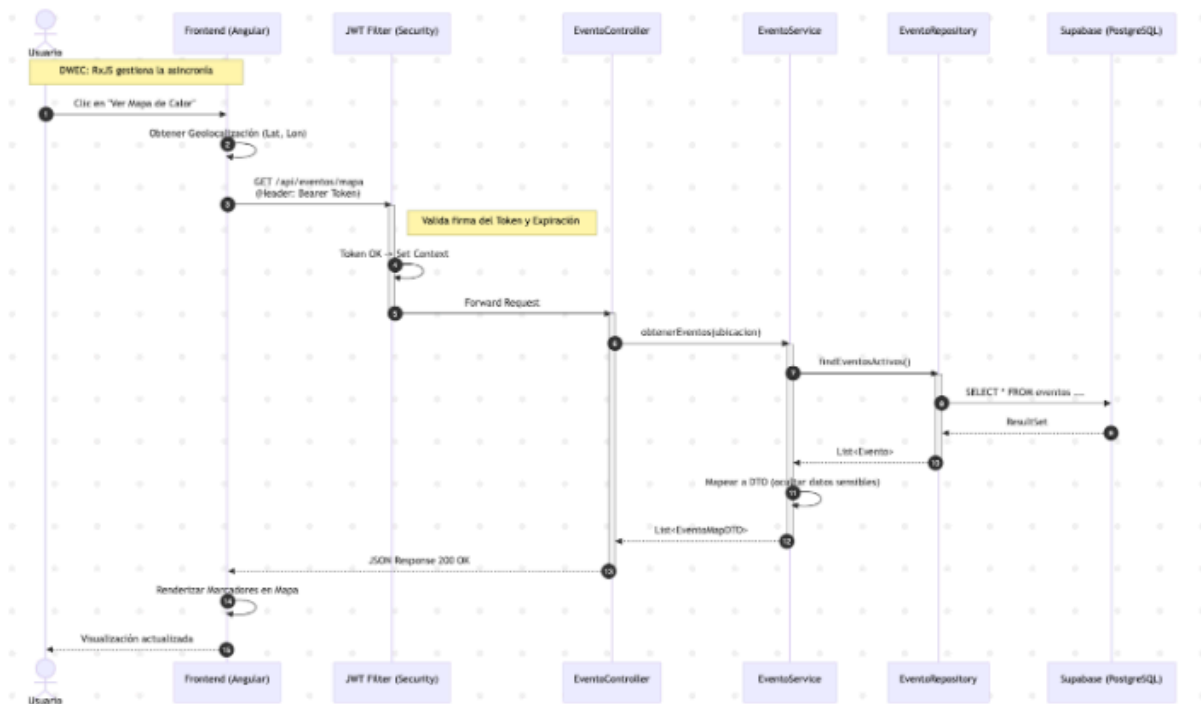
6.2.1.- Diagrama de casos de uso.



6.2.2.- Diagrama de clases.



6.2.3.- Diagrama de secuencia.



6.3.- Arquitectura utilizada.

El sistema se basa en una arquitectura cliente-servidor, donde cada capa cumple una función específica y se comunica mediante las API REST utilizando el protocolo HTTP. Este enfoque permite separar claramente la lógica de presentación, la lógica de negocio y la persistencia de datos.

La arquitectura del sistema se divide en los siguientes componentes principales:

- Cliente (Frontend):
 - El cliente consiste en una SPA (Single Page Application) desarrollada con Angular, encargada de gestionar toda la interacción con el usuario.
 - Las principales responsabilidades que tiene son:
 - Renderizar la interfaz de usuario.
 - Gestionar la navegación entre vistas.
 - Realizar peticiones HTTP al backend.
 - Procesar y mostrar los datos recibidos.
 - Gestionar eventos que realiza el usuario.
- Servidor (Backend):
 - El servidor se ha desarrollado utilizando Spring Boot, el cual actúa como un núcleo, siendo responsable de la lógica de negocio y la gestión de datos.
 - Las funciones principales que cumple son las siguientes:
 - Exponer los endpoints REST para el acceso a los datos.
 - Procesar las peticiones del cliente.
 - Validar los datos antes del almacenamiento de estos.

- Gestionar la autenticación y las autorizaciones de los usuarios.
- El backend además actúa como un intermediario entre el Frontend y la BBDD, lo cual garantiza seguridad y consistencia.
- Base de datos:
 - El sistema que se ha empleado es una base de datos relacional, la cual es PostgreSQL, diseñada para almacenar de forma estructurada toda la información de la aplicación.
 - Entre los datos gestionados se encuentran los usuarios, eventos, categorías y las relaciones entre entidades
 - El acceso hacia la base de datos se realiza mediante JPA/Hibernate, lo cual permite trabajar con entidades orientadas a objetos.
- Infraestructura (Docker):
 - La aplicación se ejecuta en un entorno basado en contenedores mediante Docker, lo cual permite encapsular cada uno de los servicios del sistema en unidades independientes.
 - Los contenedores utilizados son:
 - Contenedor con la aplicación Spring Boot.
 - Contenedor con la BBDD.
 - Las ventajas del uso de contenedores que nos ofrece Docker son:
 - Aislamiento de dependencias entre servicios.
 - Facilita el despliegue en distintos entornos.
 - Simplifica la configuración del entorno a la hora de desarrollarlo.

El flujo de funcionamiento del sistema describe cómo interactúan los distintos componentes desde que el usuario realiza cierta acción hasta que recibe una respuesta:

1. El usuario interactúa con la interfaz del frontend (por ejemplo, aplicando un filtro o seleccionando un evento).
2. El frontend realiza una petición HTTP a la API REST del backend.
3. El backend recibe la solicitud y la procesa aplicando la lógica de negocio correspondiente.
4. Si es necesario, el backend consulta la base de datos para obtener o almacenar información.
5. La base de datos devuelve los datos solicitados al backend.
6. El backend construye una respuesta en formato JSON.
7. El frontend recibe la respuesta y actualiza la interfaz de usuario dinámicamente.

Lo que permite este flujo es una comunicación eficiente y desacoplada entre los distintos componentes del sistema.

Características de la arquitectura:

- La arquitectura implementada presenta una serie de ventajas clave:
- Desacoplamiento: frontend y backend pueden evolucionar de forma independiente.

- Escalabilidad: cada componente puede ampliarse según la carga del sistema.
- Mantenibilidad: facilita la detección y corrección de errores.
- Reutilización: los endpoints pueden ser consumidos por otros clientes (apps móviles, etc.).
- Seguridad: el backend controla el acceso a los datos.
- Portabilidad: gracias a Docker, el sistema puede ejecutarse en diferentes entornos sin modificaciones.

Posibles mejoras futuras:

- Uso de balanceadores de carga para distribuir peticiones
- Implementación de caché para mejorar el rendimiento
- Despliegue en la nube con servicios escalables
- Separación en microservicios en caso de crecimiento del sistema

6.4.- Prototipos.

Los prototipos permiten representar de manera visual el cómo será la interfaz de usuario de la aplicación, facilitando la planificación del diseño y la experiencia de usuario antes del desarrollo completo. En este proyecto se han realizado sketches y wireframes como herramientas principales para poder visualizar la estructura y el flujo de la aplicación.

6.4.1.- Sketches.

Los sketches son representaciones rápidas realizadas a mano de las pantallas y funcionalidades principales. Permiten explorar ideas de diseño, distribución de elementos y el flujo de navegación de manera ágil.

El objetivo principal de ellos es definir la estructura básica de la interfaz y la interacción entre las pantallas.

Herramientas utilizadas: papel y lápiz.

Sketches aquí →

6.4.2.- Wireframes.

Los wireframes son representaciones más detalladas sobre la interfaz, mostrando la ubicación de los elementos, la jerarquía de la información y flujos de navegación, sin incluir estilos visuales definitivos.

El objetivo es planificar la UI de manera estructurada y servir de guía para el desarrollo del frontend.

Herramientas utilizadas: Figma.

Wireframes aquí →

7.- Pruebas

En el otro documento →

8.- Conclusiones

El desarrollo de Scena nos ha permitido comprender de forma práctica cómo es trabajar en equipo en un entorno de desarrollo real, donde es necesario coordinar diferentes roles, gestionar tareas y mantener una comunicación constante para alcanzar un objetivo común: el desarrollo de una aplicación web.

A lo largo del proyecto hemos adquirido experiencia en el desarrollo de una aplicación completa, abarcando tanto el frontend como el backend, así como la gestión de la base de datos y la integración de todos los componentes del sistema. Además, hemos trabajado con tecnologías actuales utilizadas en el sector profesional, lo que nos ha permitido reforzar y ampliar nuestros conocimientos técnicos.

También hemos mejorado nuestra capacidad de organización y adaptación al trabajo bajo presión, especialmente en momentos en los que ha sido necesario resolver problemas técnicos o ajustar funcionalidades dentro de los plazos establecidos.

Por otro lado, este proyecto nos ha permitido desarrollar habilidades tanto técnicas como personales, como la comunicación dentro del equipo, la toma de decisiones conjunta y la resolución de diferencias de criterio durante el desarrollo.

En conclusión, consideramos que Scena ha sido una experiencia muy completa y enriquecedora a nivel académico y personal, ya que nos ha acercado al funcionamiento real de un entorno de desarrollo profesional y nos ha preparado para afrontar proyectos de mayor complejidad en el futuro.

9.- Bibliografía:

Metodología:

Schwaber, K., & Sutherland, J. (2020). The Scrum Guide: <https://scrumguides.org/>

Frontend

Angular. (s.f.). Angular Documentation: <https://angular.io/docs>

Microsoft. (s.f.). TypeScript Handbook: <https://www.typescriptlang.org/docs/>

W3C. (s.f.). HTML5 Specification: <https://www.w3.org/TR/html52/>

W3C. (s.f.). CSS Specification: <https://www.w3.org/Style/CSS/>

Bootstrap. (s.f.). Bootstrap Documentation: <https://getbootstrap.com/docs/>

Leaflet.js. (s.f.). Leaflet Documentation: <https://leafletjs.com/>

Backend

Gosling, J., et al. (2019). The Java Language Specification. Oracle:

<https://docs.oracle.com/javase/specs/>

Pivotal / VMware. (s.f.). Spring Boot Reference Documentation.:

<https://docs.spring.io/spring-boot/docs/current/reference/html/>

Spring. (s.f.). Spring Security Reference:

<https://docs.spring.io/spring-security/site/docs/current/reference/html5/>

Jakarta EE. (s.f.). Java Persistence API (JPA) Specification:

<https://jakarta.ee/specifications/persistence/>

Hibernate. (s.f.). Hibernate ORM Documentation:

<https://hibernate.org/orm/documentation/>

Base de datos

PostgreSQL Global Development Group. (s.f.). PostgreSQL Documentation:

<https://www.postgresql.org/docs/>

Strauch, C. (2011). NoSQL Databases. Lecture Notes, University of Hamburg.

Sadalage, P. J., & Fowler, M. (2012). NoSQL Distilled. Addison-Wesley.

UX / UI

Norman, D. A. (2013). The Design of Everyday Things. Basic Books.

Nielsen, J., & Tahir, R. (2012). Usability of Websites for Mobile Devices. Nielsen

Norman Group. Recuperado de <https://www.nngroup.com/articles/mobile-usability/>

Pruebas de software

ISTQB Foundation Level Syllabus. (última edición): <https://www.istqb.org/>

Beizer, B. (1990). Software Testing Techniques (2nd ed.). Van Nostrand Reinhold.

Análisis de mercado / Estrategia / DAFO

Kotler, P., & Keller, K. L. (2015). Marketing Management (15th ed.). Pearson.

Pickton, D. W., & Wright, S. (1998). What's SWOT in Strategic Analysis?

Arquitectura de software / APIs / REST

Fielding, R. T. (2000). Architectural Styles and the Design of Network-based Software Architectures (Dissertation). UC Irvine:

<https://www.ics.uci.edu/~fielding/pubs/dissertation/>

Richardson, L., & Ruby, S. (2007). RESTful Web Services. O'Reilly.

Contenerización / Docker

Merkel, D. (2017). Docker and Container Fundamentals. Apress.

Docker. (s.f.). Docker Documentation: <https://docs.docker.com/>

Herramientas generales

Git / GitHub: <https://git-scm.com/doc>

Supabase: <https://supabase.com/docs>

Visual Studio Code: <https://code.visualstudio.com/docs>